

UNIVERSITÀ POLITECNICA DELLE MARCHE

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA  
INFORMATICA E DELL'AUTOMAZIONE

---

## Progetto Zero Trust Architecture 2026

*Protezione Adattiva di Database e API con Verifica Continua,  
Machine Learning e Micro-Segmentazione*

---

### Candidati:

Andrea Flaiani (Matr. 1126928)  
Andrea Altieri (Matr. 1128865)  
Niccolò de Pascali (Matr. 1123958)  
Matteo Risolo (Matr. 1122743)  
Simone Murazzo (Matr. 1113295)

### Docente:

Prof. Luca Spalazzi

# Indice

---

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione e obiettivi</b>                                       | <b>4</b>  |
| 1.1      | La protezione dei dati aziendali e delle risorse critiche             | 4         |
| 1.2      | La filosofia Zero Trust: “never Trust, always Verify”                 | 4         |
| 1.3      | Il caso di studio: l’infrastruttura ospedaliera                       | 5         |
| 1.4      | Obiettivi del progetto                                                | 5         |
| 1.5      | Struttura della relazione                                             | 6         |
| <b>2</b> | <b>Architettura di sistema e microsegmentazione</b>                   | <b>7</b>  |
| 2.1      | Le quattro reti segregate                                             | 7         |
| 2.2      | Il choke-point obbligato: envoy proxy                                 | 9         |
| 2.3      | La tupla multidimensionale Zero Trust (ZTA 6D)                        | 10        |
| 2.4      | Orchestrazione e Isolamento dei Container                             | 11        |
| 2.5      | Il server database di risorsa (MongoDB)                               | 11        |
| 2.6      | Le web API e il portale client di simulazione                         | 12        |
| <b>3</b> | <b>Implementazione pratica e configurazione</b>                       | <b>13</b> |
| 3.1      | La microsegmentazione e il deploy con docker-compose                  | 13        |
| 3.2      | La configurazione di Envoy Proxy come PEP                             | 13        |
| 3.3      | Il motore decisionale OPA (PDP) e l’integrazione Splunk MLTK          | 15        |
| 3.4      | Regole di network protection e tracciamento su nftables               | 17        |
| 3.5      | Autenticazione crittografica: mTLS, TPM e fingerprinting JA3          | 18        |
| 3.5.1    | Mutua autenticazione TLS (mTLS)                                       | 18        |
| 3.5.2    | Attestazione hardware TPM                                             | 19        |
| 3.5.3    | Fingerprinting software JA3                                           | 20        |
| 3.6      | Rilevamento delle intrusioni di rete: Snort NIDS                      | 20        |
| 3.7      | Aggregazione centralizzata dei log: Fluent-Bit                        | 21        |
| 3.8      | Il modello predittivo di Machine Learning: Gradient Boosting          | 22        |
| 3.8.1    | Dataset di addestramento e feature                                    | 22        |
| 3.8.2    | Addestramento e scelta dell’algoritmo                                 | 22        |
| 3.8.3    | Predizione in tempo reale                                             | 23        |
| <b>4</b> | <b>Simulazione degli scenari e validazione</b>                        | <b>24</b> |
| 4.1      | Scenario 1: accesso legittimo                                         | 24        |
| 4.2      | Scenario 2: dispositivo non autorizzato                               | 26        |
| 4.3      | Scenario 3: accesso remoto negato                                     | 29        |
| 4.4      | Scenario 4: tentativo di bypass L4 e rilevamento attivo               | 31        |
| 4.5      | Scenario 5: protezione dai payload dannosi a livello applicativo (L7) | 33        |
| 4.5.1    | Simulazione di compromissione del dispositivo (Bypass L7)             | 34        |
| <b>5</b> | <b>Conclusioni e sviluppi futuri</b>                                  | <b>37</b> |
| 5.1      | Obiettivi raggiunti                                                   | 37        |
| 5.2      | Sviluppi futuri                                                       | 37        |

# Elenco delle figure

---

|    |                                                                                                                                                                                                |    |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Topologia di rete dell'infrastruttura Zero Trust con flussi logici e di telemetria. . . . .                                                                                                    | 8  |
| 2  | Diagramma dei flussi informativi e dei componenti della Zero Trust Architecture. . . . .                                                                                                       | 9  |
| 3  | Schermata del portale di login protetto dell'ospedale San Raffaele. . . .                                                                                                                      | 24 |
| 4  | Interfaccia del database pazienti visualizzata con successo da Alice dopo validazione TPM e rischio minimo. . . . .                                                                            | 25 |
| 5  | Log di autorizzazione (ALLOW) per l'accesso di Alice indicizzato in Splunk, comprensivo del risk score comportamentale calcolato ( $\approx 9.96$ ). . . . .                                   | 25 |
| 6  | Mappatura logica dei flussi nello Scenario 1 (Accesso legittimo di Alice). . . . .                                                                                                             | 26 |
| 7  | Schermata di errore 403 Forbidden mostrata a Bob per mancata attestazione hardware del chip TPM. . . . .                                                                                       | 27 |
| 8  | Log di diniego (DENY) per l'accesso di Bob indicizzato in Splunk, evidenziante l'assenza della firma hardware TPM ( <code>tpm_present: false</code> ). . . . .                                 | 28 |
| 9  | Mappatura logica dei flussi nello Scenario 2 (Blocco al login di Bob per mancanza di TPM). . . . .                                                                                             | 28 |
| 10 | Schermata di errore 403 Forbidden mostrata a Charlie a causa del tentativo di login remoto non consentito. . . . .                                                                             | 29 |
| 11 | Log di diniego (DENY) per l'accesso remoto di Charlie indicizzato in Splunk, evidenziante la mancata conformità della localizzazione di rete ( <code>network_internal: false</code> ). . . . . | 30 |
| 12 | Mappatura logica dei flussi nello Scenario 3 (Blocco al login di Charlie da remoto). . . . .                                                                                                   | 30 |
| 13 | Schermata del pannello di ricerca di Splunk che mostra il log di blocco [NFT-BLOCK] indicizzato. . . . .                                                                                       | 32 |
| 14 | Mappatura logica dei flussi nello Scenario 4 (Tentativo di bypass e isolamento). . . . .                                                                                                       | 32 |
| 15 | Schermata di errore 403 Forbidden visualizzata a seguito del blocco di una query MongoDB dannosa a livello L7. . . . .                                                                         | 34 |
| 16 | Log di blocco (response code 403) relativo alla richiesta L7 di Alice in Splunk, che evidenzia l'intercettazione del metodo DELETE sulla risorsa <code>/api/patients</code> . . . . .          | 34 |
| 17 | Log di decisione DENY indicizzato in Splunk per il tentativo di accesso diretto al database da parte di Alice (PC compromesso), con risk score pari a $\approx 95.49$ . . . . .                | 36 |
| 18 | Mappatura logica dei flussi nello Scenario 5 (Blocco di una query MongoDB dannosa a livello L7). . . . .                                                                                       | 36 |

# Elenco dei listati

---

|    |                                                                                                                                                                                                         |    |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Definizione delle quattro reti segregate nel file docker-compose.yaml. . .                                                                                                                              | 7  |
| 2  | Definizione delle quattro reti segregate nel file docker-compose.yaml. . .                                                                                                                              | 13 |
| 3  | Estratto della configurazione del listener MongoDB in envoy.yaml. . . .                                                                                                                                 | 14 |
| 4  | Configurazione del filtro HTTP Lua in envoy.yaml per il blocco preventivo L7. . . . .                                                                                                                   | 15 |
| 5  | Struttura della query Splunk MLTK generata dinamicamente da OPA. . .                                                                                                                                    | 15 |
| 6  | Regole di ispezione profonda L7 DPI implementate nel file rules.rego. .                                                                                                                                 | 16 |
| 7  | Regole di autorizzazione adattiva basate su soglie e postura del dispositivo in rules.rego. . . . .                                                                                                     | 17 |
| 8  | Configurazione delle catene di filtering e NAT nello script di avvio del firewall. . . . .                                                                                                              | 18 |
| 9  | Generazione dei certificati client con estensione TPM opzionale. . . . .                                                                                                                                | 19 |
| 10 | Validazione dell'estensione TPM nel certificato client da parte di OPA. .                                                                                                                               | 20 |
| 11 | Regole di rilevamento personalizzate per Snort. . . . .                                                                                                                                                 | 21 |
| 12 | Query SPL per l'addestramento del modello Gradient Boosting su Splunk MLTK. . . . .                                                                                                                     | 23 |
| 13 | Simulazione di attacco bypass diretto tramite curl dal terminale di Charlie.                                                                                                                            | 31 |
| 14 | Dettaglio del log di blocco generato da nftables per il tentativo di bypass.                                                                                                                            | 31 |
| 15 | Simulazione di accesso diretto al database MongoDB dal terminale del dispositivo compromesso di Alice, utilizzando i certificati mTLS originali per autenticarsi verso Envoy sulla porta 27017. . . . . | 35 |

## CAPITOLO 1

# Introduzione e obiettivi

---

*In questo primo capitolo viene presentato il contesto applicativo e metodologico alla base del progetto, definendo le sfide di sicurezza tipiche della gestione di database distribuiti e illustrando i principi teorici del paradigma Zero Trust. Si introduce quindi il caso di studio ospedaliero scelto come dominio di riferimento e si delineano gli obiettivi e la struttura complessiva della relazione.*

### 1.1 La protezione dei dati aziendali e delle risorse critiche

Le infrastrutture IT aziendali rappresentano uno dei bersagli più ambiti e critici per gli attori malevoli a livello globale. La digitalizzazione dei servizi e la transizione verso sistemi in cloud hanno introdotto enormi benefici in termini di efficienza operativa, tra cui l'accesso condiviso a risorse sensibili, l'automazione dei flussi e il monitoraggio IoT di asset industriali. Tuttavia, questa evoluzione ha contemporaneamente esteso a dismisura la superficie d'attacco delle organizzazioni.

La criticità di questo scenario è dettata da molteplici fattori. In primo luogo, vi è la natura estremamente riservata delle informazioni trattate all'interno dei database, che includono dati finanziari, proprietà intellettuali, credenziali e profili di utenti. La perdita di riservatezza o l'alterazione di tali dati viola gravemente le principali normative vigenti sulla protezione dei dati personali e sulla sicurezza delle reti a livello internazionale. In secondo luogo, va considerato l'impatto sulla **business continuity**: a differenza dei semplici disservizi temporanei, un attacco informatico di tipo ransomware che cripta o altera i database operativi può causare il blocco completo dei servizi, enormi perdite finanziarie e danni reputazionali incalcolabili. Infine, l'eterogeneità degli accessi complica ulteriormente il quadro: gli operatori e gli amministratori devono poter consultare i dati sia dai terminali interni alla rete aziendale, come le workstation d'ufficio, sia da remoto, ad esempio in regime di smart working, introducendo vettori di minaccia esterni non controllati.

### 1.2 La filosofia Zero Trust: “never Trust, always Verify”

La sicurezza perimetrale classica, basata sul modello a “castello”, in cui un firewall protegge una rete interna considerata intrinsecamente sicura da una rete esterna ostile, si è dimostrata obsoleta e inefficace. Una volta che un attaccante riesce a penetrare il perimetro, tramite phishing, exploit o **insider threat**, ottiene totale libertà di movimento laterale.

Il paradigma **Zero Trust Architecture** (ZTA), codificato dal NIST nel documento speciale *SP 800-207*, supera questa concezione abbattendo il concetto di fiducia implicita basata sulla localizzazione di rete. I cardini della filosofia ZTA sono tre:

1. **Mai fidarsi, verificare sempre (Never Trust, Always Verify)**: ogni singola richiesta di accesso ad una risorsa, indipendentemente dalla rete da cui proviene, sia essa interna o esterna, deve essere autenticata, autorizzata e validata crittograficamente prima dell'inoltro.

2. **Accesso con privilegi minimi (Least Privilege):** l'accesso viene concesso in modo estremamente granulare e ristretto solo alle risorse necessarie per lo svolgimento della specifica attività, ad esempio limitando la scrittura e la cancellazione di record sensibili ai soli operatori autorizzati.
3. **Presumere la violazione (Assume Breach):** progettare il sistema assumendo che alcuni segmenti siano già compromessi. Ciò impone l'adozione di micro-segmentazione di rete per limitare il raggio d'azione dell'attaccante, crittografia end-to-end per proteggere i dati in transito e sistemi di monitoraggio continuo del traffico.

### 1.3 Il caso di studio: l'infrastruttura ospedaliera

Il dominio applicativo scelto per la validazione dell'architettura è quello di una **struttura ospedaliera**, un contesto in cui la criticità dei dati trattati e l'eterogeneità degli accessi rendono particolarmente evidente la necessità di un approccio Zero Trust.

Il sistema gestisce un database MongoDB contenente informazioni sanitarie altamente sensibili, tra cui anagrafiche dei pazienti, cartelle cliniche e dati diagnostici. L'accesso a queste risorse è richiesto da tre categorie di operatori con profili di rischio differenti:

- **Alice:** personale medico interno all'ospedale, dotato di workstation aziendale con chip **TPM** hardware e collegato dalla rete locale (10.0.1.0/24). Rappresenta l'utente a rischio minimo.
- **Bob:** operatore interno che utilizza un dispositivo personale privo di attestazione hardware TPM. Pur collegandosi dalla rete aziendale, la mancanza di garanzie sull'integrità del dispositivo lo sottopone a soglie di rischio più restrittive.
- **Charlie:** professionista in regime di smart working, dotato di certificato e chip TPM ma collegato da una rete esterna non controllata (192.168.100.0/24). La provenienza da una rete non fidata impone politiche di tolleranza al rischio analogamente restrittive.

Questo scenario consente di dimostrare concretamente come l'architettura adatti dinamicamente le proprie soglie di sicurezza in funzione del contesto multidimensionale di ciascun accesso, combinando l'identità dell'utente, la postura del dispositivo e la localizzazione di rete.

### 1.4 Obiettivi del progetto

Il presente progetto si pone l'obiettivo di realizzare un'architettura Zero Trust completa, funzionante ed integrata per la protezione di un database MongoDB aziendale. Il sistema è progettato per operare una valutazione continua e adattiva del rischio basata sul contesto dell'utente e del suo dispositivo.

Per raggiungere questo scopo, vengono impiegate diverse tecnologie integrate. In primo luogo, envoy opera come **Policy Enforcement Point** (PEP), intercettando ogni singola richiesta HTTP o MongoDB in modo sicuro mediante crittografia mutua TLS (*mTLS*) ed eseguendo il fingerprinting software (*JA3*). In secondo luogo, open policy agent (OPA) funge da **Policy Decision Point** (PDP) decentralizzato, valutando le

richieste sulla base delle sei dimensioni Zero Trust (ZTA 6D), rappresentate dai parametri  $u$ ,  $d$ ,  $s$ ,  $n$ ,  $a$ ,  $r$ . Il monitoraggio e la correlazione sono affidati a `splunk` SIEM, che integra i log aggregati e calcola un punteggio di rischio dinamico in tempo reale tramite modelli predittivi di machine learning basati sull'algoritmo **Gradient Boosting**. Infine, a livello di rete, `nftables` e `snort` (NIDS) garantiscono rispettivamente la microsegmentazione del traffico a livello L3/L4 e il rilevamento tempestivo di tentativi di movimento laterale o bypass del proxy.

### 1.5 Struttura della relazione

Il documento è organizzato come segue: il Capitolo 2 illustra l'architettura complessiva della topologia di rete a quattro zone. Il Capitolo 3 scende nel dettaglio dell'implementazione tecnica di ciascun componente di sicurezza, descrivendo il funzionamento di `envoy`, `OPA`, `splunk`, `snort` e `nftables`. Il Capitolo 4 descrive la validazione del sistema attraverso la simulazione di scenari d'attacco ed accessi legittimi. Infine, il Capitolo 5 riassume i risultati e propone futuri sviluppi progettuali.

## CAPITOLO 2

# Architettura di sistema e microsegmentazione

*In questo capitolo viene analizzata l'architettura logica e fisica dell'infrastruttura, descrivendo la segmentazione di rete a quattro zone e il ruolo di envoy come unico punto di transito autorizzato per il traffico dati.*

### 2.1 Le quattro reti segregate

La sicurezza dell'infrastruttura si basa sul principio della **microsegmentazione**, implementata definendo quattro reti virtuali isolate tramite Docker. Questa compartimentazione riduce drasticamente l'area di impatto di un'eventuale compromissione. La segmentazione fisica ed i relativi indirizzamenti IP delle quattro aree protette sono schematizzati visivamente nella Figura 1, mentre la loro definizione a livello di infrastruttura è dettagliata nella configurazione mostrata nel Listato 1.

```
docker-compose.yaml

1 networks:
2   frontend-net:
3     driver: bridge
4     ipam:
5       config:
6         - subnet: 10.0.1.0/24
7   backend-net:
8     driver: bridge
9   control-plane-net:
10    driver: bridge
11   external-net:
12     driver: bridge
13     ipam:
14       config:
15         - subnet: 192.168.100.0/24
```

**Listing 1:** Definizione delle quattro reti segregate nel file docker-compose.yaml.

La prima area è la **zona untrusted**, che rappresenta la rete esterna non controllata da cui provengono le richieste di utenti remoti. In questa zona opera il client dell'operatore in smart working. Non vi è alcuna rotta di instradamento che colleghi direttamente questa zona al database o ai servizi interni: l'unico punto di contatto esposto è il listener di envoy protetto da crittografia.

La seconda area è la **zona DMZ o frontend**, assimilabile alla rete locale fisica dell'azienda. Qui risiedono i client interni utilizzati dal personale d'ufficio e il firewall nftables. Anche in questo caso, i client del frontend non possiedono le rotte di rete



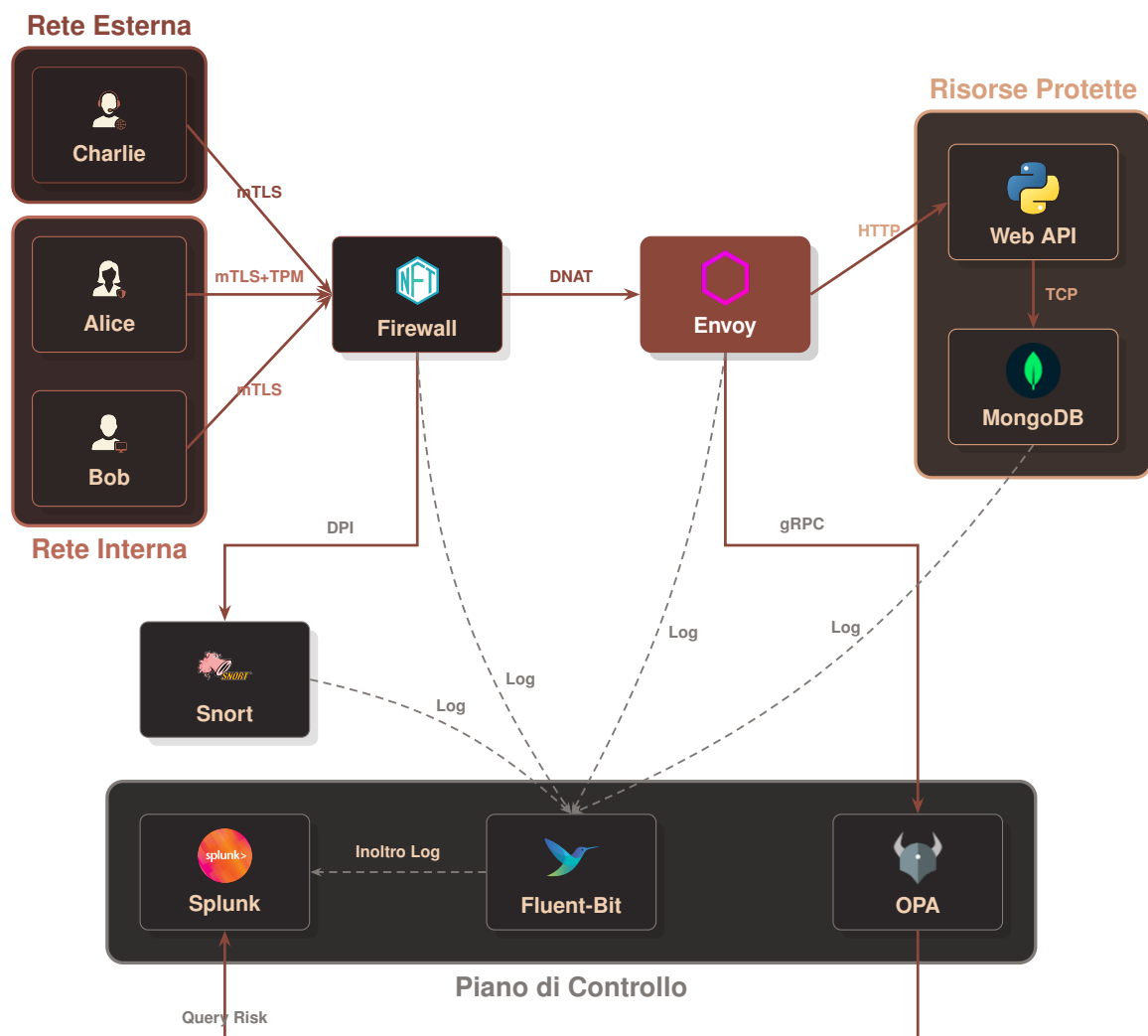


Figura 1: Topologia di rete dell'infrastruttura Zero Trust con flussi logici e di telemetria.

per raggiungere il backend: il firewall intercetta tutto il traffico a livello L3/L4 e devia le richieste autorizzate verso il proxy.

La terza area è il **secure core o backend**, una rete altamente protetta e completamente isolata. All'interno di questa zona sono collocati il database mongodb e le Web API del servizio aziendale. Nessun client esterno o interno può avviare una connessione diretta con queste macchine: l'unico container autorizzato a inoltrare pacchetti nel backend è il proxy, che agisce da intermediario per le sole richieste esplicitamente validate. In merito alla scelta del database, le specifiche del progetto consentivano l'adozione di mongodb o, in alternativa, di una configurazione composta da proxysql e mysql. In questa architettura si è preferito utilizzare il database orientato ai documenti mongodb per agevolare l'ispezione granulare dei comandi BSON nativi direttamente a livello applicativo, sfruttando le capacità di decodifica del protocollo del proxy.

La quarta e ultima area è la **control plane o trust zone**. Si tratta di una rete di gestione isolata in cui risiedono i sistemi decisionali e di monitoraggio, tra cui il motore delle regole OPA, il raccoglitore di log fluent-bit e il SIEM splunk. Questa zona comunica esclusivamente con il proxy per la validazione delle richieste e con tutti i container per la raccolta della telemetria in background.

## 2.2 Il choke-point obbligato: envoy proxy

Al centro dell'intera topologia si colloca envoy, che agisce come **Policy Enforcement Point (PEP)** e rappresenta il punto di strozzatura obbligato per tutto il traffico. Qualsiasi tentativo di comunicazione che cerchi di scavalcare questo passaggio viene intercettato e scartato a livello di rete dal firewall. Il posizionamento strategico del PEP e il flusso dei messaggi di autorizzazione verso gli altri moduli sono illustrati nella Figura 2.

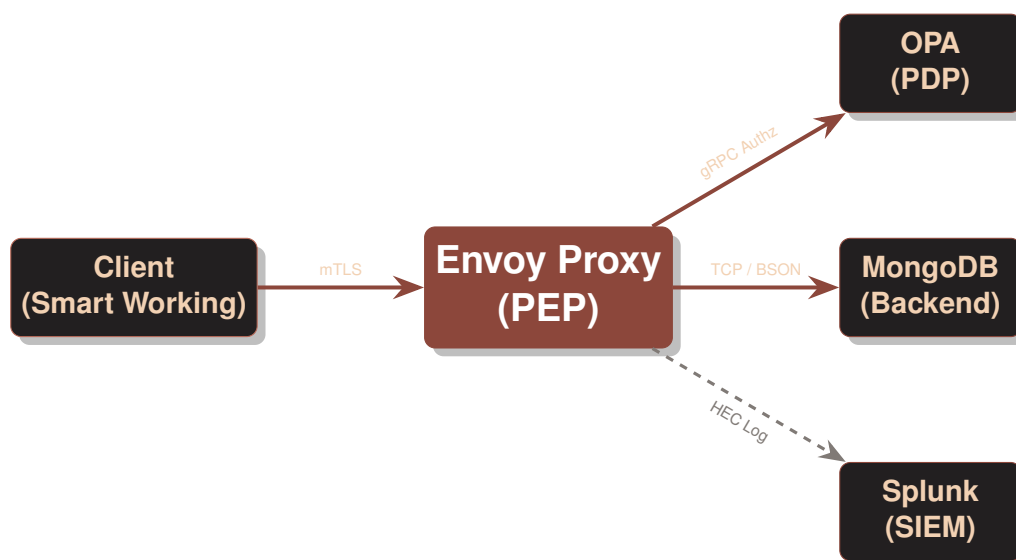


Figura 2: Diagramma dei flussi informativi e dei componenti della Zero Trust Architecture.

Il funzionamento del proxy prevede la separazione dei flussi su porte distinte per gestire in modo sicuro i diversi protocolli. Sulla porta 8443 viene gestito il traffico HTTP cifrato per le Web API aziendali, mentre sulla porta 27017 transita il traffico TCP nativo diretto al database. Per ogni connessione, il proxy avvia una negoziazione crittografica

pretendendo la presentazione di un certificato client valido. Durante questa fase, viene calcolato l'hash della firma del software client per determinare il fingerprint *JA3*.

Prima di inoltrare la richiesta verso il backend, il proxy sospende la sessione e interroga OPA tramite una chiamata gRPC. Solo dopo aver ricevuto un responso positivo dal PDP, il proxy provvede a stabilire la connessione con la risorsa richiesta nel backend, garantendo l'applicazione del paradigma Zero Trust per ogni singola transazione.

Questa separazione dei ruoli rispecchia una precisa stratificazione a livello della pila protocollare ISO/OSI. Mentre il firewall `nftables` opera a livello di rete e trasporto (L3/L4), intercettando e filtrando i pacchetti in base agli indirizzi IP e alle porte TCP/UDP, il proxy envoy lavora fino al livello applicativo (L7) e di sessione/presentazione (TLS), consentendo una validazione profonda e contestuale che non sarebbe possibile con un tradizionale filtro di rete a pacchetti.

## 2.3 La tupla multidimensionale Zero Trust (ZTA 6D)

Il nucleo del motore decisionale risiede nella formalizzazione matematica e logica del contesto di accesso attraverso una tupla a sei dimensioni, definita come:

$$T = (u, d, s, n, a, r) \quad (1)$$

Ciascuna dimensione mappa un attributo specifico estratto dinamicamente a livello L4-L7 e valutato in tempo reale per determinare l'affidabilità della richiesta:

1. **Utente** ( $u \in U$ ): l'identità forte del soggetto richiedente, estratta dal campo `Subject` del certificato client X.509 presentato durante la fase di handshake mTLS (`DOWNSTREAM_PEER_SUBJECT`). Questa dimensione identifica univocamente gli operatori (*Alice, Bob, Charlie*).
2. **Dispositivo** ( $d \in D$ ): lo stato di postura dell'hardware. Viene validato verificando la presenza di un'estensione X.509 personalizzata con Object Identifier (OID) specifico 1.3.6.1.4.1.9999.1. Tale estensione attesta che la chiave privata del client sia custodita all'interno di un chip **TPM (Trusted Platform Module)** hardware o emulato, discriminando tra workstation sicure autorizzate e dispositivi non censiti.
3. **Software** ( $s \in S$ ): il fingerprint crittografico della suite client, calcolato tramite l'algoritmo **JA3**. Questo consente a Envoy di identificare il software effettivo che avvia la connessione (es. un browser autorizzato vs uno script Python malevolo), prevenendo attacchi basati su spoofing dell'User-Agent.
4. **Contesto di Rete** ( $n \in N$ ): la provenienza geografica o topologica del pacchetto, classificata in base all'indirizzo IP di origine (`DOWNSTREAM_REMOTE_ADDRESS`). Permette di distinguere le richieste provenienti dalla rete locale interna ospedaliera (10.0.1.0/24) da quelle provenienti dall'esterno (Smart Working tramite la subnet 192.168.100.0/24).
5. **Azione** ( $a \in A$ ): l'operazione richiesta. Nel caso del protocollo HTTP corrisponde al metodo (es. GET, POST), mentre per le connessioni database MongoDB viene estratta a livello L7 dal filtro `mongo_proxy` (es. comandi di find, insert, update o delete).

6. **Risorsa** ( $r \in R$ ): il target dell'azione, rappresentato dall'URI del servizio API HTTP richiesto o dal nome della collezione MongoDB (es. `patients`, `medical-records`) estratta dinamicamente tramite ispezione profonda dei pacchetti (DPI).

## 2.4 Orchestrazione e Isolamento dei Container

L'intera architettura è stata implementata e ingegnerizzata utilizzando **Docker** e **Docker Compose**, per garantire la massima portabilità e un isolamento rigoroso a livello di sistema operativo. Come mostrato nel `docker-compose.yaml`, l'infrastruttura è composta da diversi container, ognuno con un ruolo specifico e confinato in base al principio del minimo privilegio.

Le reti virtuali agiscono come veri e propri *VLAN tagging* all'interno dello stack Docker. Ad esempio, il container `zta-mongodb` è collegato esclusivamente alla rete `backend-net`, rendendolo inaccessibile a chiunque non possieda un'interfaccia sulla stessa rete (come `envoy`). Analogamente, il `zta-firewall` espone un'interfaccia su `frontend-net`, `backend-net` ed `external-net`, fungendo da gateway per instradare o dropare i pacchetti ancor prima che raggiungano il livello applicativo.

L'isolamento non è limitato solo alla rete, ma si estende ai namespace e ai privilegi: il container `zta-snort` (IDS) condivide lo stesso *network namespace* di `Envoy` (`network_mode: "service:envoy-pep"`), permettendogli di intercettare passivamente tutto il traffico che transita per il proxy, senza però avere la possibilità di interferire o bloccare direttamente la comunicazione.

## 2.5 Il server database di risorsa (MongoDB)

Il nucleo di memorizzazione persistente dei dati clinici, denominato *secure core*, è costituito dal database orientato ai documenti `mongodb`, ospitato nel container `zta-mongodb` e isolato interamente all'interno della rete privata `backend-net`. Come descritto nello script di inizializzazione `init-mongo.js`, il database `hospital_db` definisce tre collezioni principali preposte alla gestione delle cartelle cliniche e dello stato di trust dell'infrastruttura.

La collezione `patients` memorizza le cartelle cliniche dei pazienti. Ciascun documento contiene dati anagrafici e clinici di base come `patient_id`, `name`, `age`, `ward` (reparto di ricovero), `blood_type` (gruppo sanguigno) e, in particolare, i campi altamente confidenziali `sensitive_notes` (note cliniche ad alta sensibilità) e `treatment` (piano terapeutico). La riservatezza di queste informazioni è tutelata da controlli applicativi L7, i quali scattano qualora un utente cerchi di estrarre note cliniche in condizioni contestuali non conformi. La collezione `identities` funge invece da ZTA Policy Store a livello di database. Questa memorizza le identità censite associando ciascun principal SPIFFE (es. `spiffe://zta.hospital/ns/default/sa/client-alice`) al rispettivo identificatore del dispositivo client, al ruolo applicativo assegnato (es. `medico`) e a parametri di fiducia di base (`trust_score` e `trust_baseline`), utilizzati per la misurazione continua della reputazione. Infine, la collezione `trust_history` registra lo storico delle variazioni del livello di fiducia dell'utente in seguito ad accessi consentiti o tentativi anomali bloccati dal PDP, consentendo analisi forensi ed auditing continuo.

Dal punto di vista dell'accesso di rete, il container del database non espone alcuna porta verso l'esterno né è direttamente raggiungibile dai client. Qualsiasi interrogazione deve transitare obbligatoriamente attraverso il modulo proxy `envoy` che intercet-

ta le chiamate sulla porta standard 27017, decodifica il protocollo BSON ed esegue l'enforcement delle decisioni restituite da OPA.

## 2.6 Le web API e il portale client di simulazione

Il frontend dell'applicazione e la logica di business sono stati ingegnerizzati per simulare l'interazione quotidiana in un ambiente ospedaliero reale e validare l'efficacia del perimetro Zero Trust.

Il backend applicativo è implementato in Python tramite il framework FastAPI nel container `zta-web-api`. Oltre a fungere da ponte verso il database MongoDB per l'estrazione delle cartelle cliniche tramite gli endpoint `/api/patients` e `/api/patients/-sensitive`, il servizio agisce come **Policy Information Point (PIP)** per il calcolo del rischio comportamentale. Ricevuta la richiesta di autorizzazione da OPA, il backend arricchisce la query inserendovi le feature comportamentali dinamiche (es. tentativi di login falliti e frequenza di sessione rilevata da un tracker interno) ed interroga il SIEM Splunk MLTK per ottenere la predizione del rischio.

Il portale web a cui accedono i medici è erogato in modo indipendente per ciascun client simulato (`client-alice`, `client-bob`, `client-charlie`) tramite container dedicati che eseguono un server web locale (`simulator.py`) e presentano pagine web scritte in HTML/CSS moderno. I tre client simulano diverse posture di sicurezza. L'utente Alice rappresenta un medico operante da una workstation d'ufficio interna autorizzata dotata di chip TPM e attestazione hardware valida sulla subnet `10.0.1.0/24`. Alice accede correttamente al portale, visualizzando la dashboard verde e potendo consultare i dati clinici standard e sensibili dei pazienti. Il client di Bob descrive invece l'accesso tramite un dispositivo personale sprovvisto di attestazione crittografica TPM, che OPA intercetta bloccando la sessione fin dal login e mostrando a schermo una pagina di errore 403 personalizzata. Infine, il client di Charlie simula un tentativo di connessione da una rete esterna non protetta in modalità smart working sulla subnet `192.168.100.0/24`, la quale viene bloccata al login a causa delle restrizioni di rete.

Per convalidare sul campo le politiche impostate, il sistema integra funzionalità per testare la robustezza della ZTA. Per quanto riguarda gli attacchi applicativi a livello L7, la barra di ricerca del portale intercetta input contenenti parole chiave distruttive come `DROP` o `DELETE`, generando una richiesta HTTP anomala che viene bloccata dal filtro Lua di Envoy o da OPA. Invece, i tentativi di bypass a livello L4 vengono eseguiti direttamente dal terminale del container client (es. tramite un comando `curl` lanciato dal terminale di riga di comando verso le porte interne del database o delle API). In questo scenario, le regole del firewall `nftables` scartano istantaneamente i pacchetti non transitati da Envoy, e il sistema IDS Snort notifica l'evento al SIEM Splunk.

## CAPITOLO 3

# Implementazione pratica e configurazione

*In questo capitolo viene analizzata l'implementazione pratica dell'architettura Zero Trust, dettagliando le configurazioni dei singoli container e i meccanismi di interazione tra i moduli di controllo e protezione del traffico.*

### 3.1 La microsegmentazione e il deploy con docker-compose

Il deploy dell'infrastruttura è orchestrato tramite la tecnologia Docker, definendo quattro reti virtuali distinte e isolate nel file `docker-compose.yaml`. Questa separazione impedisce il routing diretto tra i client esterni o di frontend e il secure core di backend, forzando il transito del traffico attraverso il proxy Envoy. La configurazione delle reti isolate è definita nel file mostrato nel Listato 2.

```
1 networks:
2   frontend-net:
3     driver: bridge
4     ipam:
5       config:
6         - subnet: 10.0.1.0/24
7   backend-net:
8     driver: bridge
9   control-plane-net:
10    driver: bridge
11   external-net:
12     driver: bridge
13     ipam:
14       config:
15         - subnet: 192.168.100.0/24
```

**Listing 2:** Definizione delle quattro reti segregate nel file `docker-compose.yaml`.

### 3.2 La configurazione di Envoy Proxy come PEP

Il proxy envoy funge da Policy Enforcement Point (PEP) ed è configurato per intercettare e autenticare in modo trasparente e sicuro tutto il traffico dati. La configurazione del proxy, definita nel file `envoy.yaml`, implementa due listener principali:

- **Listener database (mongodb\_listener):** opera sulla porta 27017 e impone l'uso del protocollo mTLS obbligatorio per tutti i client che tentano di connettersi al database MongoDB.

- **Listener HTTP (`http_listener`):** opera sulla porta 8443 per il traffico diretto alle Web API ospedaliere.

Per estrarre le metriche necessarie alla valutazione del rischio e all'ispezione DPI, Envoy utilizza una catena di filtri specializzati. In particolare, il filtro `tls_inspector` esegue il calcolo in tempo reale dell'impronta digitale *JA3* durante la negoziazione TLS, mentre il filtro `mongo_proxy` analizza i comandi applicativi eseguiti sui socket TCP. L'integrazione con OPA è gestita tramite il filtro `ext_authz`, il quale sospende la sessione TCP o HTTP per inviare un payload gRPC contenente i metadati estratti e i certificati client al PDP. Un estratto significativo della configurazione di Envoy per il listener del database è mostrato nel Listato 3.

```

1 static_resources:
2   listeners:
3     - name: mongodb_listener
4       address:
5         socket_address:
6           address: 0.0.0.0
7           port_value: 27017
8       listener_filters:
9         - name: envoy.filters.listener.tls_inspector
10          typed_config:
11            "@type": type.googleapis.com/envoy.extensions.filters.
12              listener.tls_inspector.v3.TlsInspector
13            enable_ja3_fingerprinting: true
14          filter_chains:
15            - filters:
16              - name: envoy.filters.network.mongo_proxy
17                typed_config:
18                  "@type": type.googleapis.com/envoy.extensions.filters.
19                    network.mongo_proxy.v3.MongoProxy
20                  stat_prefix: mongodb
21                  emit_dynamic_metadata: true
22            - name: envoy.filters.network.ext_authz
23              typed_config:
24                "@type": type.googleapis.com/envoy.extensions.filters.
25                  network.ext_authz.v3.ExtAuthz
26                stat_prefix: OPA_authz
27                transport_api_version: V3
28                grpc_service:
29                  envoy_grpc:
30                    cluster_name: OPA_cluster
31                  timeout: 45s
32                include_peer_certificate: true

```

**Listing 3:** Estratto della configurazione del listener MongoDB in `envoy.yaml`.

Nel listener HTTP, prima di interpellare OPA ed inoltrare la richiesta alle Web API di backend, viene inserito un filtro L7 basato sul motore Lua (`envoy.filters.http.lua`). Questo modulo funge da firewall applicativo locale in grado di bloccare sul nascere richieste a percorsi sensibili o metodi distruttivi non consentiti a livello applicativo (es.



comandi di tipo DELETE), sgravando il PDP e i sistemi di backend dalla gestione di chiamate palesemente malevole. La configurazione di questo componente è riportata nel Listato 4.

#### envoy.yaml (Filtro HTTP Lua)

```

1 - name: envoy.filters.http.lua
2   typed_config:
3     "@type": type.googleapis.com/envoy.extensions.filters.http.lua.v3
      .Lua
4     inline_code: |
5       function envoy_on_request(request_handle)
6         local path = request_handle:headers():get(":path")
7         local method = request_handle:headers():get(":method")
8
9         -- L7 DPI firewall rules
10        if path == "/api/patients/sensitive" then
11          request_handle:respond({[":status"] = "403"}, '{"error": "
L7 Firewall Block: Access to sensitive notes forbidden"}')
12        elseif method == "DELETE" and string.match(path, "^/api/
patients") then
13          request_handle:respond({[":status"] = "403"}, '{"error": "
L7 Firewall Block: Destructive operations (DROP) are forbidden"}')
14        end
15      end

```

**Listing 4:** Configurazione del filtro HTTP Lua in envoy.yaml per il blocco preventivo L7.

### 3.3 Il motore decisionale OPA (PDP) e l'integrazione Splunk MLTK

Il Policy Decision Point (PDP) riceve la tupla  $T$  da Envoy ed esegue la valutazione delle regole scritte in Rego all'interno del file `rules.rego`. Per supportare decisioni adattive basate sul contesto, OPA invia una query predittiva sincrona via HTTP all'API predittiva di Splunk MLTK.

La query SPL, mostrata nel Listato 5, viene assemblata dinamicamente con le variabili contestuali correnti ed inviata al SIEM per ottenere il punteggio di rischio aggiornato dell'utente.

#### Splunk SPL Query (Dynamically Generated by OPA)

```

1 | makeresults
2 | eval user="Alice", software="JA3_HASH", device="Workstation
   Ospedaliera (TPM)", network="10.0.1.5", action="find", resource="
   patients"
3 | apply trust_model
4 | rename "predicted(rischio)" as rischio
5 | table rischio

```

**Listing 5:** Struttura della query Splunk MLTK generata dinamicamente da OPA.



Accanto alla valutazione adattiva, OPA implementa controlli di sicurezza L7 stringenti (DPI) per intercettare query MongoDB dannose prima che raggiungano il backend, come mostrato nel frammento di codice del Listato 6.

```
rules.rego

1 # L7 DPI: Regole di blocco esplicite per query pericolose
2 default l7_dpi_block := false
3 l7_dpi_block := true if {
4     # Blocca query che contengono termini sensibili o distruttivi
5     contains(lower(db_query), "sensitive_notes")
6 } else := true if {
7     contains(lower(db_query), "dropdatabase")
8 } else := true if {
9     contains(lower(db_query), "deleteall")
10 }
```

**Listing 6:** Regole di ispezione profonda L7 DPI implementate nel file rules.rego.

Il motore decisionale definisce l'ammissibilità dell'accesso combinando lo stato di conformità del dispositivo, la localizzazione di rete e la valutazione dinamica dello score di rischio ricavato da Splunk. Come descritto nel Listato 7, vengono stabilite soglie di tolleranza differenziate per mitigare la superficie di minaccia contestuale: un dispositivo interno provvisto di chip TPM beneficia di una soglia di rischio elevata (pari a 50), mentre l'assenza del modulo hardware (con ripiegamento sul solo fingerprinting JA3) o la provenienza da una rete esterna riducono la tolleranza ad un valore massimo di 8. Inoltre, l'accesso alle funzioni di autenticazione viene preventivamente inibito se la variabile `is_auth_blocked` assume valore vero, bloccando le richieste di login prive di chip TPM o originate al di fuori della rete aziendale.

## rules.rego (Regole di autorizzazione adattiva)

```

1 # La regola di autorizzazione adattiva (Risk-Based & JA3 Fallback)
2 risk_ok := true if {
3     is_internal_network
4     is_tpm
5     splunk_risk_score <= 50
6 } else := true if {
7     is_internal_network
8     not is_tpm
9     software != "Sconosciuto"
10    splunk_risk_score <= 8
11 } else := true if {
12     not is_internal_network
13     is_tpm
14     splunk_risk_score <= 8
15 } else := false
16
17 # Blocca l'autenticazione al login se manca il TPM o se si e' all'
    esterno
18 is_auth_blocked := true if {
19     is_auth_request
20     not is_tpm
21 } else := true if {
22     is_auth_request
23     not is_internal_network
24 } else := false
25
26 allow := true if {
27     risk_ok
28     not l7_dpi_block
29     not is_auth_blocked
30     # ... (generazione log di successo)
31 }

```

**Listing 7:** Regole di autorizzazione adattiva basate su soglie e postura del dispositivo in rules.rego.

### 3.4 Regole di network protection e tracciamento su nftables

Il firewall, basato sulla tecnologia nftables, blinda la rete forzando l'instradamento obbligatorio verso il proxy Envoy e gestendo una trappola attiva per rilevare e bloccare i tentativi di bypass.

All'avvio del container tramite lo script start.sh, il firewall esegue quattro compiti principali:

1. **Risoluzione dinamica:** interroga il DNS interno di Docker per ottenere l'IP del container Envoy (zta-envoy).
2. **Port forwarding L4:** reindirizza tutto il traffico TCP legittimo diretto alla porta del servizio verso l'IP di Envoy sulla porta 8443 tramite regola DNAT e applica il mascheramento SNAT.

3. **Trappola di bypass:** intercetta i tentativi di connessione diretta alle porte di backend non autorizzate (es. porta 8000), invia i dettagli del pacchetto al demone di logging `ulogd2` con il prefisso `[NFT-BLOCK]` e scarta istantaneamente il traffico (drop).
4. **Denylist dinamica:** crea un set di indirizzi IP bloccati, gestito in tempo reale da un'API di management in Python che consente di aggiungere o rimuovere IP in base alle anomalie rilevate.

La configurazione delle catene e delle regole nftables applicate all'avvio del firewall è mostrata nel Listato 8.

```

start.sh
1 # --- Inizializzazione Base NFTables ---
2 nft flush ruleset
3 nft add table ip filter
4 nft add set ip filter denylist { type ipv4_addr \; }
5 nft add chain ip filter forward { type filter hook forward priority 0
6   \; policy accept \; }
7 # Regole di DROP per gli IP bannati
8 nft add rule ip filter forward ip saddr @denylist counter drop
9
10 # --- Port Forwarding (DNAT) verso Envoy ---
11 nft add table ip nat
12 nft add chain ip nat prerouting { type nat hook prerouting priority
13   -100 \; }
14 nft add chain ip nat postrouting { type nat hook postrouting priority
15   100 \; }
16 nft add rule ip nat prerouting tcp dport 8443 counter dnat to
17   $ENVOY_IP:8443
18 nft add rule ip nat postrouting ip daddr $ENVOY_IP tcp dport 8443
19   counter masquerade
20
21 # --- TRAPPOLA: Log e Drop del traffico diretto al backend ---
22 nft add rule ip nat prerouting tcp dport 8000 log group 0 prefix "[
23   NFT-BLOCK] " counter
24 nft add rule ip nat prerouting tcp dport 8000 drop

```

**Listing 8:** Configurazione delle catene di filtering e NAT nello script di avvio del firewall.

### 3.5 Autenticazione crittografica: mTLS, TPM e fingerprinting JA3

L'architettura implementa un meccanismo di verifica dell'identità a tre livelli complementari, che operano in modo congiunto durante la fase di handshake TLS per costruire le dimensioni *u* (utente), *d* (dispositivo) e *s* (software) della tupla Zero Trust.

#### 3.5.1 Mutua autenticazione TLS (mTLS)

Ogni listener di Envoy impone la presentazione obbligatoria di un certificato client X.509 firmato dalla CA interna dell'ospedale (ZTA Hospital Trust Root CA). La direttiva `require_client_certificate: true` nella configurazione TLS di Envoy garantisce

che nessuna connessione priva di certificato valido possa superare la fase di negoziazione. Il campo Common Name (CN) del certificato identifica univocamente l'operatore (es. employee-alice, employee-bob) e viene propagato ad OPA tramite il metadato `DOWNSTREAM_PEER.SUBJECT`.

### 3.5.2 Attestazione hardware TPM

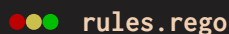
Per discriminare i dispositivi aziendali sicuri dai terminali personali non censiti, il sistema utilizza un'estensione X.509 personalizzata con Object Identifier (OID) 1.3.6.1.4.1.9999.1. Durante la generazione dei certificati tramite lo script `generate_identities.py`, i certificati destinati a workstation dotate di chip TPM (come Alice e Charlie) vengono arricchiti con questa estensione, il cui contenuto attesta crittograficamente il legame tra la chiave privata e l'hardware. Il frammento di generazione è mostrato nel Listato 9.

```
generate_identities.py

1 TPM_OID = ObjectIdentifier("1.3.6.1.4.1.9999.1")
2
3 def generate_leaf(base_dir, name, cn, ca_cert, ca_key, has_tpm=False)
4     :
5     key = rsa.generate_private_key(public_exponent=65537, key_size
6     =2048)
7     subject = x509.Name([x509.NameAttribute(NameOID.COMMON_NAME, cn)
8     ])
9     builder = x509.CertificateBuilder()
10    # ... (configurazione base del certificato)
11    if has_tpm:
12        builder = builder.add_extension(
13            x509.UnrecognizedExtension(
14                TPM_OID,
15                b"TPM-VERIFIED-HARDWARE-ROOT-OF-TRUST-PCR0-OK"
16            ), critical=False
17        )
18    cert = builder.sign(ca_key, hashes.SHA256())
```

**Listing 9:** Generazione dei certificati client con estensione TPM opzionale.

A livello di policy, OPA estrae e valida questa estensione in tempo reale decodificando il certificato PEM ricevuto da Envoy e verificando la presenza dell'OID specifico nell'array delle estensioni, come mostrato nel Listato 10.



```

1 # C. DISPOSITIVO E TPM
2 default is_tpm := false
3 is_tpm := true if {
4     cert_raw := input.attributes.source.certificate
5     cert_pem := urlquery.decode(cert_raw)
6     certs := crypto.x509.parse_certificates(cert_pem)
7     count(certs) > 0
8     some ext in certs[0].Extensions
9     ext.Id == [1, 3, 6, 1, 4, 1, 9999, 1]
10 }
11 default device := "Dispositivo non censito (No TPM)"
12 device := "Workstation Ospedaliera Sicura (TPM Validato)" if { is_tpm
    }

```

**Listing 10:** Validazione dell'estensione TPM nel certificato client da parte di OPA.

Questo approccio garantisce la proprietà di non esportabilità della chiave privata del client: poiché la chiave viene generata all'interno del chip TPM del dispositivo, essa non può essere copiata o trasferita su una macchina differente. Di conseguenza, il certificato utente viene legato in modo indissolubile all'hardware autorizzato.

### 3.5.3 Fingerprinting software JA3

Il terzo livello di identificazione opera a livello di trasporto, senza richiedere alcuna collaborazione da parte del client. Durante l'handshake TLS, il filtro `tls_inspector` di Envoy calcola l'hash **JA3**, un'impronta digitale deterministica basata sulla combinazione dei parametri del *ClientHello* (versione TLS, cipher suite offerte, estensioni, curve ellittiche e formati dei punti). Poiché ogni software client genera un *ClientHello* caratteristico, l'hash JA3 consente di distinguere un browser autorizzato (es. Firefox o Chrome) da uno strumento di attacco (es. curl o nmap), anche in presenza di spoofing dell'header User-Agent. L'hash viene reso disponibile come metadato dinamico e propagato ad OPA sia per il listener MongoDB (tramite i metadati del `tls_inspector`) sia per il listener HTTP (tramite l'header iniettato `x-client-fingerprint`).

Tuttavia, l'impiego del solo fingerprinting software presenta un limite strutturale sul piano della sicurezza: se due dispositivi distinti utilizzano lo stesso browser (o la medesima versione di client HTTP) ed eseguono lo stesso sistema operativo, la negoziazione TLS produrrà un hash JA3 identico. In questo scenario, i dispositivi diventano indistinguibili per il proxy, evidenziando come il solo fingerprinting software non possa sostituire le garanzie di postura offerte dall'attestazione hardware.

## 3.6 Rilevamento delle intrusioni di rete: Snort NIDS

A completamento delle difese applicative, l'architettura integra snort come **Network Intrusion Detection System** (NIDS) passivo, posizionato strategicamente in modo da intercettare tutto il traffico che transita per il proxy.

La tecnica di deploy adottata è quella della condivisione del *network namespace*: nel file `docker-compose.yaml`, il container `zta-snort` è configurato con la direttiva `network_mode: "service:envoy-pep"`, che gli consente di osservare passivamente tutti i

pacchetti in ingresso e in uscita dall'interfaccia di rete di Envoy senza possedere un proprio indirizzo IP e senza poter alterare o bloccare il traffico.

All'avvio, Snort viene lanciato in ascolto sull'interfaccia principale del namespace condiviso, con output dei log nella directory `/var/log/snort` condivisa tramite volume Docker con Fluent-Bit. Le regole personalizzate implementate nel file `local.rules` definiscono due firme di rilevamento principali, mostrate nel Listato 11.

```

●●● local.rules

1 # 1. Rileva una scansione Nmap (SYN Scan) verso Envoy o DB
2 alert tcp any any -> any any (msg:"ZTA ALERT: Rilevata Scansione
3   di Rete (Possibile Nmap)"; flags:S; threshold:type both,
4   track by_src, count 20, seconds 10; sid:1000001; rev:1;)
5
6 # 2. Rileva un tentativo di connessione diretta (bypass) a MongoDB
7 alert tcp any any -> any 27017 (msg:"ZTA ALERT: Tentativo di
8   accesso diretto a MongoDB bypassando Envoy!";
9   sid:1000002; rev:1;)

```

**Listing 11:** Regole di rilevamento personalizzate per Snort.

La prima regola implementa un rilevamento basato su soglia (*threshold*): se vengono osservati più di 20 pacchetti SYN dallo stesso IP sorgente in un intervallo di 10 secondi, viene generato un allarme per sospetta scansione di rete. La seconda regola rileva qualsiasi tentativo di connessione TCP diretto alla porta 27017 di MongoDB che, per architettura, dovrebbe essere raggiungibile esclusivamente dal proxy Envoy.

### 3.7 Aggregazione centralizzata dei log: Fluent-Bit

La raccolta e la centralizzazione dei log di telemetria provenienti dai diversi componenti dell'infrastruttura è affidata a `fluent-bit`, un agente di log forwarding leggero e ad alte prestazioni. `Fluent-Bit` è stato preferito al classico `syslog` per la sua capacità nativa di operare in ambienti containerizzati, supportando il parsing strutturato di log JSON e l'invio diretto verso Splunk tramite il protocollo **HTTP Event Collector** (HEC).

La configurazione, definita nel file `fluent-bit.conf`, implementa una pipeline di raccolta multi-sorgente con cinque canali di input e un'unica destinazione di output, come schematizzato nella Tabella 1.

Tabella 1: Pipeline di raccolta log di Fluent-Bit: sorgenti e tag associati.

| Sorgente          | Tag             | Percorso Volume                | Host Iniettato |
|-------------------|-----------------|--------------------------------|----------------|
| Snort IDS         | snort.alerts    | /var/log/snort/alert_json.txt  | zta-snort      |
| MongoDB           | mongo.audit     | /var/log/mongodb/mongod.log    | zta-mongodb    |
| Envoy PEP         | envoy.access    | /var/log/envoy/access.log      | zta-envoy      |
| OPA PDP           | opa.decisions   | /var/log/opa/decision.log      | zta-opa        |
| NFTables Firewall | nftables.kernel | /var/log/nftables/firewall.log | zta-firewall   |

L'architettura sfrutta i **volumi Docker condivisi** come meccanismo di trasporto: ogni container scrive i propri log in un volume dedicato (es. `envoy_logs`, `snort_logs`) e

Fluent-Bit li legge in modalità `tail` dal percorso montato. Per i log strutturati di OPA, Fluent-Bit applica una catena di filtri dedicata: un primo filtro `grep` seleziona solo le righe contenenti il marcatore `[OPA-PDP]`, un parser con espressione regolare estrae il payload JSON dalla stringa di log, e un secondo parser deserializza il JSON strutturato contenente la decisione, il risk score e le sei dimensioni ZTA.

Tutti i log vengono infine inoltrati verso l'unica destinazione: il **Splunk SIEM** tramite il modulo di output `splunk` configurato con il token HEC e la connessione TLS verso la porta 8088 del container `splunk-siem`.

### 3.8 Il modello predittivo di Machine Learning: Gradient Boosting

Il calcolo del punteggio di rischio dinamico rappresenta il cuore della valutazione adattiva e continua dell'architettura Zero Trust. Il modello predittivo è implementato all'interno di `splunk` tramite il **Machine Learning Toolkit** (MLTK) e utilizza l'algoritmo **Gradient Boosting Regressor**.

#### 3.8.1 Dataset di addestramento e feature

Il modello viene addestrato su un dataset sintetico di 10.000 record generati dallo script `generate_simulated_traffic.py`. Il dataset riproduce un mix realistico di traffico ospedaliero, con l'80% di sessioni legittime e il 20% di sessioni anomale o malevole. Ogni record è descritto da **12 feature** suddivise in due categorie:

- **6 feature identitarie (ZTA 6D)**: corrispondenti alle dimensioni della tupla  $T = (u, d, s, n, a, r)$  estratte in tempo reale da Envoy.
- **6 feature comportamentali**: calcolate e iniettate in tempo reale dal proxy Web-API prima di inoltrare la query a Splunk. Esse comprendono il numero di *login falliti* nelle ultime 24 ore, l'*ora della richiesta*, un flag per *accesso notturno* (22:00–06:00), la *frequenza di sessione* nell'ultima ora, il *livello di sensibilità* della risorsa (da 1 a 3) e i *giorni di inattività* dall'ultimo accesso riuscito.

Il valore target rischio (da 0 a 100) è calcolato deterministicamente come somma pesata di penalità cumulative, secondo la formula:

$$\text{Rischio} = \Delta_{\text{HW}} + \Delta_{\text{Rete}} + \Delta_{\text{Login}} + \Delta_{\text{Orario}} + \Delta_{\text{Freq}} + \Delta_{\text{Sens}} + \Delta_{\text{Inatt}} + \Delta_{\text{Azione}} + \Delta_{\text{SW}} + \epsilon \quad (2)$$

dove  $\Delta_{\text{HW}} = +20$  per dispositivi senza TPM,  $\Delta_{\text{Rete}} = +15$  per IP esterni,  $\Delta_{\text{Login}} = \min(8 \times \text{failed\_logins}, 40)$ , e  $\epsilon \sim \mathcal{N}(0, 3)$  rappresenta un rumore gaussiano per evitare l'overfitting su regole rigide.

#### 3.8.2 Addestramento e scelta dell'algoritmo

L'addestramento avviene direttamente nell'interfaccia di Splunk tramite la query SPL mostrata nel Listato 12.

## Splunk SPL | Addestramento del Trust Model

```
1 | inputlookup simulated_traffic.csv
2 | fit GradientBoostingRegressor "rischio"
3   from user, software, device, network, action, resource,
4   failed_logins, hour_of_day, is_night,
5   session_freq, sensitivity_level, days_inactive
6   into trust_model
```

**Listing 12:** Query SPL per l'addestramento del modello Gradient Boosting su Splunk MLTK.

La scelta del **Gradient Boosting** rispetto al più comune *Random Forest* è motivata dalla capacità dell'algoritmo di catturare le interazioni non lineari tra feature categoriche e numeriche. A differenza del bagging, in cui gli alberi sono indipendenti e la predizione è la media delle risposte, il boosting costruisce alberi in sequenza, dove ciascun nuovo albero è addestrato specificamente per correggere i residui (gli errori) dell'albero precedente. Questo approccio risulta particolarmente efficace nel dominio ZTA, dove il rischio non dipende da singole variabili isolate ma dalla loro combinazione: ad esempio, un utente con 5 login falliti di giorno dall'ufficio ha un rischio moderato, mentre lo stesso utente con 5 login falliti alle 3 di notte da un IP esterno rappresenta una minaccia critica.

### 3.8.3 Predizione in tempo reale

Durante il funzionamento operativo, OPA assembla dinamicamente una query SPL di tipo `apply` contenente i valori correnti delle 6 dimensioni ZTA e la invia tramite la Web-API che funge da proxy verso l'endpoint REST di Splunk. La Web-API arricchisce la richiesta con le 6 feature comportamentali calcolate in tempo reale. Il modello `trust_model` restituisce un punteggio numerico continuo (es. 5.2, 42.7, 89.1) che OPA utilizza per valutare l'accesso secondo le soglie adattive definite nelle policy Rego.



## CAPITOLO 4

# Simulazione degli scenari e validazione

Per verificare la robustezza dell'infrastruttura Zero Trust progettata, sono stati simulati cinque scenari operativi principali, ognuno mirato a testare specifiche componenti difensive e logiche del sistema. Tutti gli utenti ed operatori si interfacciano inizialmente con lo stesso portale web centralizzato, protetto da autenticazione forte e controlli contestuali (mostrato in Figura 3).

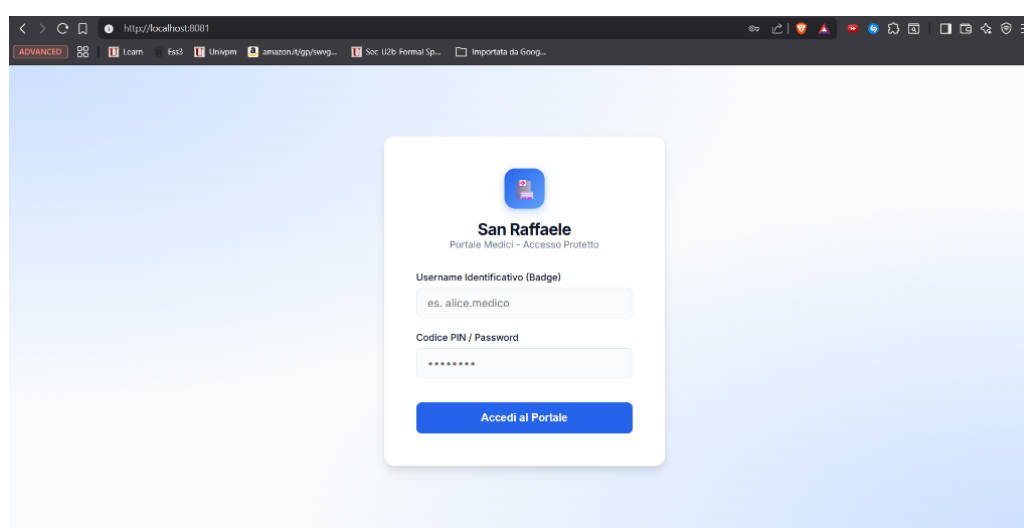
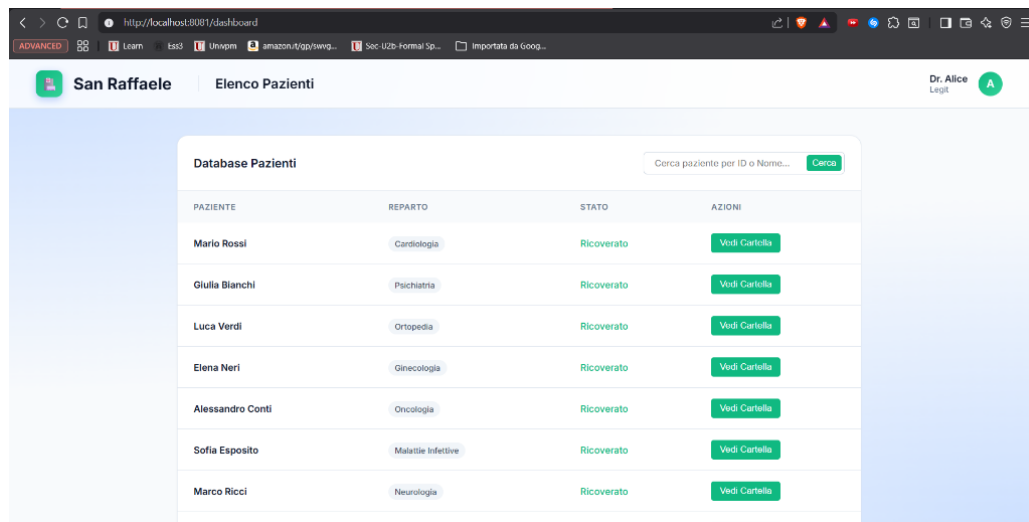


Figura 3: Schermata del portale di login protetto dell'ospedale San Raffaele.

### 4.1 Scenario 1: accesso legittimo

Il primo scenario analizza il comportamento del sistema di fronte all'accesso di un utente autorizzato dotato di dispositivo aziendale integro, simulato dall'utente Alice.

Alice effettua la connessione dall'interno della rete aziendale presentando un certificato client contenente l'estensione crittografica del chip TPM. All'arrivo della richiesta, envoy estrae il certificato e calcola il fingerprinting *JA3* del browser, trasmettendo i dati ad OPA. OPA contatta la Web API di splunk che calcola un punteggio di rischio minimo, pari a circa uno, data la totale assenza di anomalie comportamentali. Poiché il rischio è ampiamente inferiore alla soglia massima di tolleranza di cinquanta prevista per i dispositivi sicuri interni, OPA risponde con un responso positivo di **ALLOW** ed envoy instrada in pochissimi millisecondi la query verso il database, restituendo ad Alice i risultati richiesti con successo (mostrati in Figura 4). Il relativo log di OPA, indicizzato in Splunk, conferma l'esito positivo dell'operazione, la postura integra del dispositivo e il rischio complessivo calcolato (Figura 5).



| PAZIENTE         | REPARTO            | STATO      | AZIONI        |
|------------------|--------------------|------------|---------------|
| Mario Rossi      | Cardiologia        | Ricoverato | Vedi Cartella |
| Giulia Bianchi   | Psichiatria        | Ricoverato | Vedi Cartella |
| Luca Verdi       | Ortopedia          | Ricoverato | Vedi Cartella |
| Elena Neri       | Ginecologia        | Ricoverato | Vedi Cartella |
| Alessandro Conti | Oncologia          | Ricoverato | Vedi Cartella |
| Sofia Esposito   | Malattie Infettive | Ricoverato | Vedi Cartella |
| Marco Ricci      | Neurologia         | Ricoverato | Vedi Cartella |

Figura 4: Interfaccia del database pazienti visualizzata con successo da Alice dopo validazione TPM e rischio minimo.

```
> 02/06/26 17:26:14.892 { [-]
  Decision: ALLOW
  command: GET
  device: Workstation Ospedaliera Sicura (TPM Validato)
  host: zta-opa
  l7_dpi_block: false
  network_internal: true
  network_ip: 10.0.1.3
  query:
    resource: /api/patients
    risk_score: 9.961186756843691
    software: 86dab2109182b6bbaa644647d7db2997
    tpm_present: true
    user: alice
  }
  Mostra come testo non elaborato
  host = splunk-siem:8088 host = zta-opa | index = main | linecount = 1 | put
```

Figura 5: Log di autorizzazione (ALLOW) per l'accesso di Alice indicizzato in Splunk, comprensivo del risk score comportamentale calcolato ( $\approx 9.96$ ).

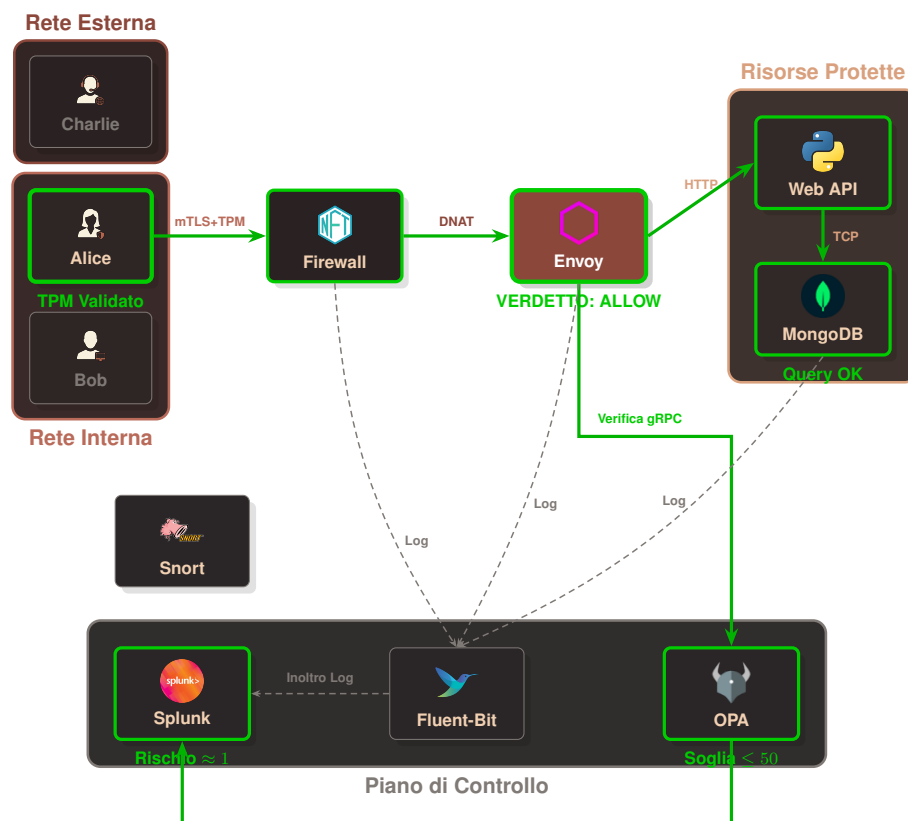


Figura 6: Mappatura logica dei flussi nello Scenario 1 (Accesso legittimo di Alice).

## 4.2 Scenario 2: dispositivo non autorizzato

Il secondo scenario testa la reazione della politica di sicurezza di fronte a un tentativo di accesso con credenziali valide ma condotto tramite un dispositivo personale privo di chip TPM hardware, simulato dall'utente Bob.

Bob riesce a connettersi alla rete ospedaliera locale con il proprio computer personale e, conoscendo le credenziali di Alice, tenta di effettuare l'autenticazione. Tuttavia, il certificato presentato dal suo dispositivo è privo dell'estensione di attestazione hardware TPM. All'arrivo della richiesta di login (`is_auth_request`), Envoy inoltra i dettagli del certificato ad OPA. Il PDP rileva la mancanza della firma hardware (`not is_tpm`) e, in conformità alle regole di sicurezza impostate, definisce la variabile di blocco dell'autenticazione (`is_auth_blocked`) come `true`. Di conseguenza, la richiesta di Bob viene rigettata sul nascere con un responso di **DENY** immediato al login, impedendogli l'accesso alla sessione ed a qualsiasi risorsa protetta, mostrando a schermo una pagina di diniego dell'accesso (Figura 7). Il relativo log di blocco registrato in Splunk (Figura 8) evidenzia la componente chiave dell'approccio Zero Trust: nonostante Bob presenti un punteggio di rischio estremamente basso ( $\approx 5.5$ , inferiore al punteggio di Alice nello Scenario 1), l'accesso viene comunque negato a causa della mancata conformità hardware del dispositivo (`tpm_present: false`). Ciò dimostra come la postura del dispositivo costituisca un vincolo forte e non compensabile da altri fattori contestuali favorevoli.

Dal punto di vista dell'architettura di rete e del modello di riferimento ISO/OSI, è opportuno analizzare i dettagli tecnici di questa restrizione. La connessione a livello di trasporto (Livello 4 - TCP) e la negoziazione del canale crittografico sicuro (Livelli 5 e 6

- TLS / mTLS) tra il client e il proxy Envoy vengono completate con successo, in quanto Bob possiede chiavi e certificati validi firmati dalla CA interna. Il diniego effettivo viene applicato esclusivamente al Livello 7 (Applicazione): il proxy Envoy, agendo da Policy Enforcement Point (PEP), analizza la richiesta HTTP, decodifica il certificato client X.509 e inoltra i relativi metadati ad OPA (PDP). È OPA che, valutando le asserzioni di policy sul certificato ad alto livello, impone il rifiuto logico. Il firewall di rete L3/L4 (nftables) non interviene in questa fase poiché l'IP e la porta di destinazione rientrano nei flussi di transito consentiti sulla rete locale. Si tratta quindi di una decisione di sicurezza di livello applicativo basata su criteri d'integrità contestuale dell'host.

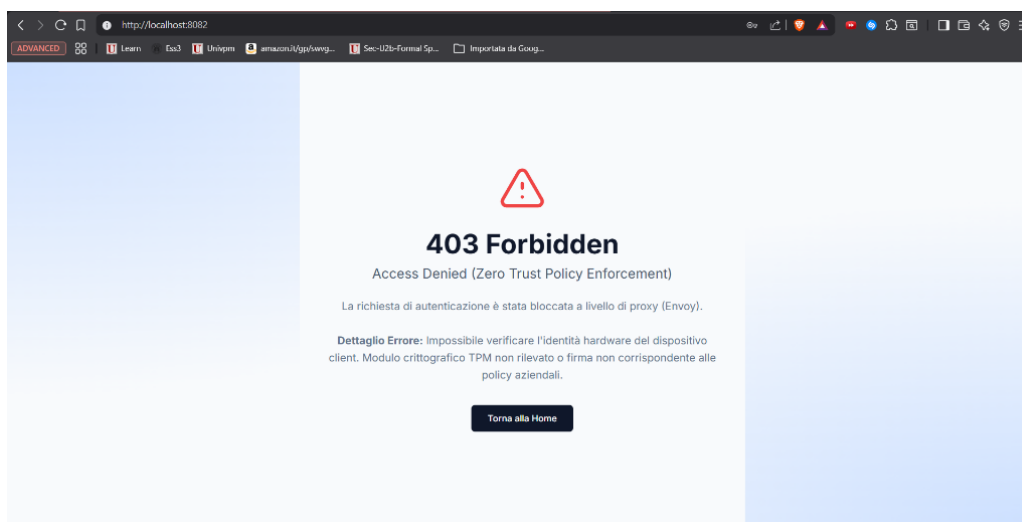


Figura 7: Schermata di errore 403 Forbidden mostrata a Bob per mancata attestazione hardware del chip TPM.

```

> 02/06/26 { [-]
17:24:35.409 Decision: DENY
               command: POST
               device: Dispositivo non censito (No TPM)
               host: zta-opa
               l7_dpi_block: false
               network_internal: true
               network_ip: 10.0.1.4
               resource: /api/auth
               risk_ok: true
               risk_score: 5.496037163707557
               software: 86dab2109182b6bbaa644647d7db2997
               tpm_present: false
               user: bob
           }
           Mostra come testo non elaborato
           host = splunk-siem:8088 host = zta-opa index = main line

```

Figura 8: Log di diniego (DENY) per l'accesso di Bob indicizzato in Splunk, evidenziante l'assenza della firma hardware TPM (tpm\_present: false).

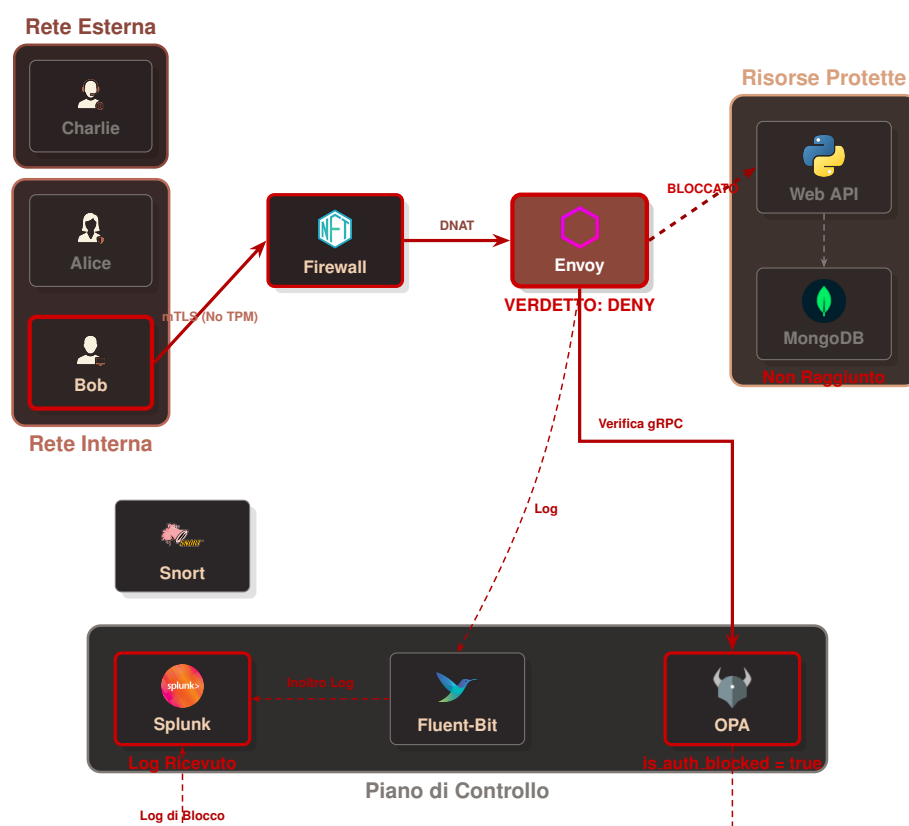


Figura 9: Mappatura logica dei flussi nello Scenario 2 (Blocco al login di Bob per mancanza di TPM).

### 4.3 Scenario 3: accesso remoto negato

Il terzo scenario analizza la gestione dell'accesso remoto (Smart Working). Charlie è un operatore legittimo dell'ospedale, dotato di credenziali valide e di un dispositivo autorizzato provvisto di chip TPM, che tenta di connettersi da casa (rete esterna non protetta).

Quando Charlie cerca di autenticarsi, OPA applica le regole restrittive per gli accessi remoti: poiché la richiesta di login proviene da una rete esterna (`not is_internal_network`), la policy ne impone il blocco preventivo impostando lo stato di `is_auth_blocked` a `true`. Di conseguenza, la richiesta di autenticazione di Charlie viene respinta immediatamente al login da Envoy su ordine di OPA, impedendo l'accesso remoto per motivi di sicurezza legati al contesto di rete, come illustrato in Figura 10. Il relativo log di blocco registrato in Splunk (Figura 11) attesta che, nonostante il dispositivo presenti una firma TPM corretta (`tpm_present: true`) ed un punteggio di rischio basso ( $\approx 5.5$ ), OPA nega l'accesso impostando il verdetto di **DENY** a causa del contesto di rete esterno (`network_internal: false`).

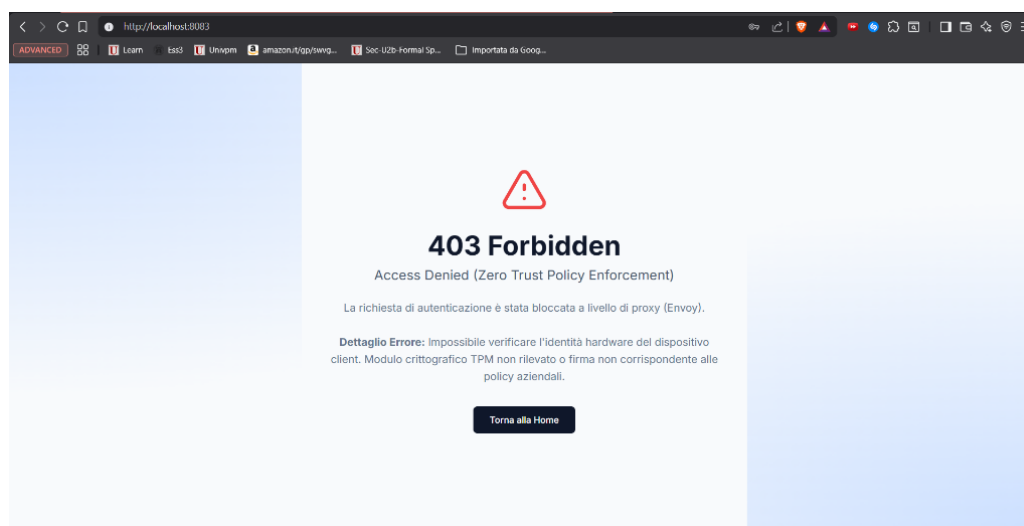


Figura 10: Schermata di errore 403 Forbidden mostrata a Charlie a causa del tentativo di login remoto non consentito.

```

> 02/06/26 { [-]
17:24:58.605 Decision: DENY
              command: POST
              device: Workstation Ospedaliera Sicura (TPM Validato)
              host: zta-opa
              l7_dpi_block: false
              network_internal: false
              network_ip: 192.168.100.3
              resource: /api/auth
              risk_ok: true
              risk_score: 5.505830381406453
              software: 86dab2109182b6bbaa644647d7db2997
              tpm_present: true
              user: charlie
            }
            Mostra come testo non elaborato
            host = splunk-siem:8088 host = zta-opa | index = main | linecount =

```

Figura 11: Log di diniego (DENY) per l'accesso remoto di Charlie indicizzato in Splunk, evidenziante la mancata conformità della localizzazione di rete (network\_internal: false).

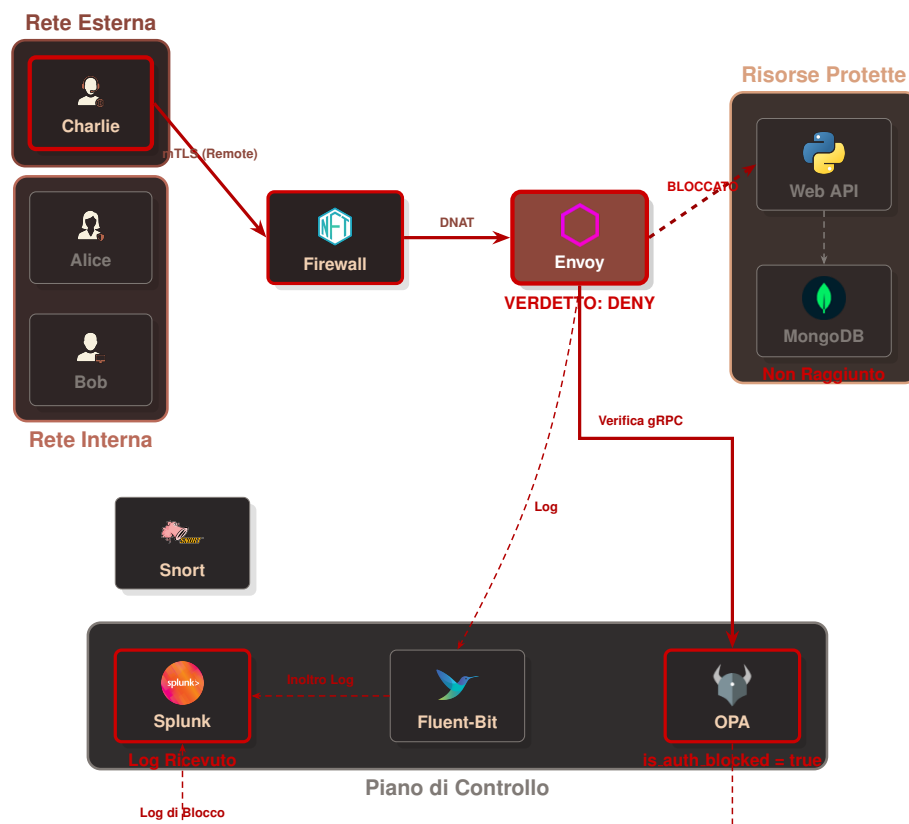


Figura 12: Mappatura logica dei flussi nello Scenario 3 (Blocco al login di Charlie da remoto).

#### 4.4 Scenario 4: tentativo di bypass L4 e rilevamento attivo

Il quarto scenario simula un tentativo di attacco in cui un utente esterno (o un dispositivo compromesso nella rete esterna, come Charlie) tenta di aggirare completamente il proxy Envoy (PEP) per connettersi direttamente al database o alle API di backend.

L'attaccante esegue una richiesta diretta tramite lo strumento `curl` dal proprio terminale verso la porta delle API:

`curl_bypass.sh`

```
1 $ docker exec -it zta-client-charlie curl -v --connect-timeout 3 http
   ://zta-firewall:8000/api/patients
2 * Host zta-firewall:8000 was resolved.
3 * Trying 192.168.100.4:8000...
4 * Connection timed out after 3002 milliseconds
5 * closing connection #0
6 curl: (28) Connection timed out after 3002 milliseconds
```

**Listing 13:** Simulazione di attacco bypass diretto tramite `curl` dal terminale di Charlie.

Come previsto, la connessione va in timeout poiché entra in azione il firewall. `nftables` rileva la connessione diretta non autorizzata e applica la regola della trappola attiva, eseguendo il DROP silente dei pacchetti. Tale evento viene registrato nei log del firewall (mostrati nel Listato 14), evidenziando l'etichetta `[NFT-BLOCK]` e tutti i parametri del pacchetto TCP bloccato sulla porta di destinazione 8000 (DPT) proveniente dal client (`SRC=10.0.1.4`).

Si osservi che, in questo scenario di bypass di rete a livello di trasporto, lo score di rischio di Splunk non viene calcolato né registrato all'interno dei log di decisione di OPA. La connessione viene infatti intercettata e interrotta a livello di rete e trasporto (Livelli 3 e 4 del modello ISO/OSI) ad opera del firewall di rete `nftables`. Poiché i pacchetti vengono scartati (DROP) in kernel space prima di poter raggiungere il proxy Envoy (PEP) e attivare la catena di autorizzazione applicativa di OPA, non si innesca alcuna chiamata al proxy delle Web API ed al modello di Machine Learning. Di conseguenza, l'unica traccia dell'evento ostile in Splunk sarà limitata alle telemetrie di blocco del firewall (`[NFT-BLOCK]`) ed agli allarmi NIDS generati da Snort.

`nftables.log`

```
1 Jun  2 17:03:04 f020514ab776 [NFT-BLOCK] IN=eth2 OUT= MAC=e6:27:66:
   d6:b7:c9:06:c9:5e:11:ef:c1:08:00 SRC=10.0.1.4 DST=10.0.1.5 LEN=60
   TOS=00 PREC=0x00 TTL=64 ID=15717 DF PROTO=TCP SPT=58384 DPT=8000
   SEQ=1187313041 ACK=0 WINDOW=64240 SYN URGP=0 MARK=0
```

**Listing 14:** Dettaglio del log di blocco generato da `nftables` per il tentativo di bypass.

I log del firewall, insieme alle telemetrie di sistema, vengono raccolti e centralizzati in tempo reale da Fluent-Bit ed inoltrati al SIEM Splunk. La Figura 13 mostra la



schermata del pannello di ricerca di Splunk Search & Reporting, in cui si può riscontrare l'avvenuta indicizzazione dell'evento di blocco [NFT-BLOCK] generato dall'host zta-firewall.

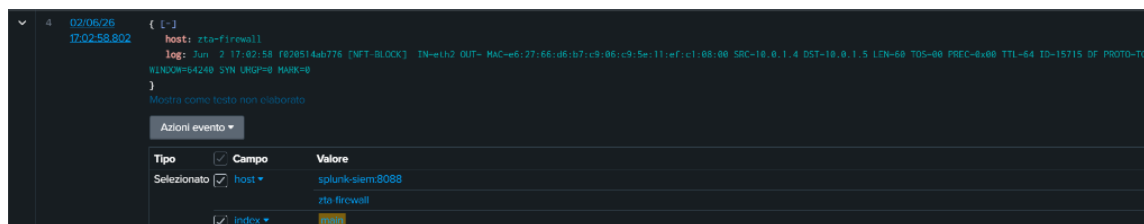


Figura 13: Schermata del pannello di ricerca di Splunk che mostra il log di blocco [NFT-BLOCK] indicizzato.

Contemporaneamente, la sonda NIDS *snort* intercetta la firma del pacchetto di attacco ed esegue l'ispezione DPI, generando un allarme di intrusione. I log di alert vengono centralizzati da *Fluent-Bit* ed inoltrati a *Splunk SIEM*, il quale imposta al valore massimo il rischio e segnala ad *OPA* di inserire permanentemente l'IP sorgente nella denylist di blocco della rete.

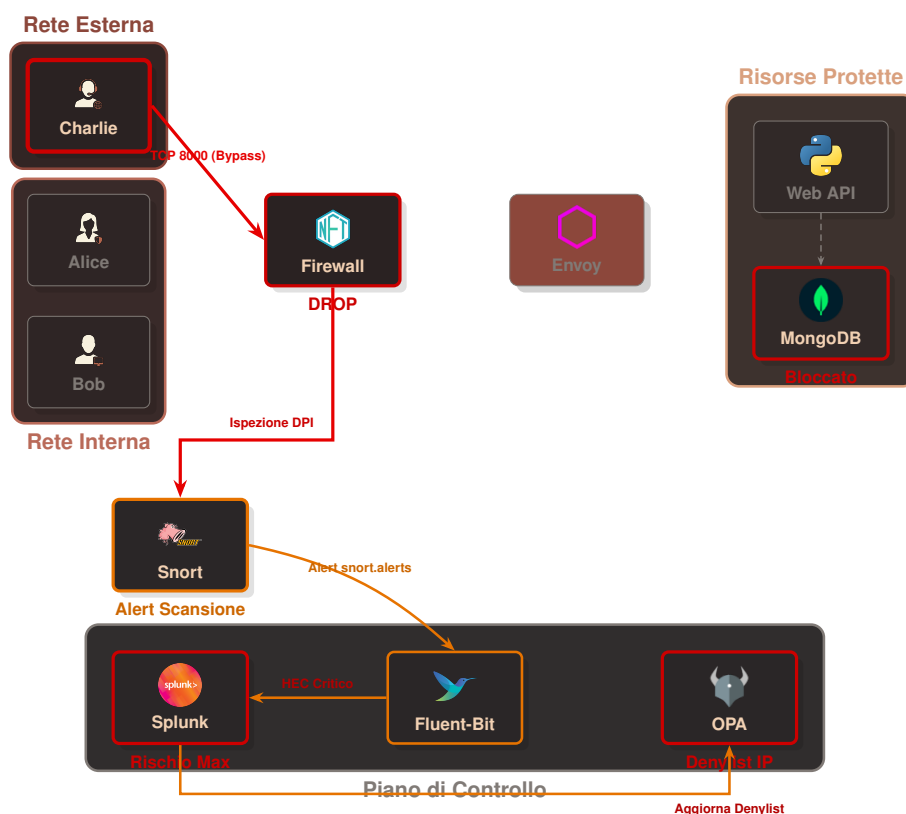


Figura 14: Mappatura logica dei flussi nello Scenario 4 (Tentativo di bypass e isolamento).

## 4.5 Scenario 5: protezione dai payload dannosi a livello applicativo (L7)

Il quinto scenario analizza il livello di sicurezza applicativo (L7) del reverse proxy Envoy. Si ipotizza il caso in cui un utente legittimo e autorizzato (come Alice, dotata di attestazione TPM valida e autenticata con successo nel sistema) sia vittima di un attacco, o che un malintenzionato tenti di eseguire comandi distruttivi o non autorizzati sul database tramite l'interfaccia di ricerca web.

Nel test effettuato, l'utente simula una SQL/NoSQL Injection inserendo la parola chiave DROP (o comandi simili come `deleteall`) nel campo di ricerca dell'interfaccia pazienti, con l'obiettivo di alterare o eliminare collezioni nel database MongoDB.

Il comportamento del calcolo del rischio adattivo presenta una distinzione tecnica fondamentale a seconda del canale utilizzato per condurre il test di injection:

- **Simulazione tramite interfaccia Web (HTTP):** Quando l'utente inserisce parole chiave dannose nel form della dashboard, il client esegue una richiesta HTTP DELETE sulla risorsa `/api/patients`. Il proxy Envoy intercetta il traffico a livello applicativo tramite il filtro `envoy.filters.http.lua` configurato localmente. Tale modulo agisce da WAF statico locale e, identificando il pattern DELETE su risorsa protetta, risponde istantaneamente con un codice 403 Forbidden. Poiché il blocco avviene prematuramente a livello di PEP (Envoy) per ottimizzare le risorse ed evitare latenze inutili, la richiesta non raggiunge mai il filtro di autorizzazione esterna (`ext_authz`); di conseguenza, OPA non viene interpellato e lo score di rischio Splunk non viene calcolato per questo flusso HTTP.
- **Simulazione tramite connessione diretta a MongoDB:** Se l'attaccante tenta di bypassare il gateway HTTP per inviare comandi direttamente sulla porta nativa del DBMS MongoDB (27017), Envoy delega la decisione ad OPA tramite il filtro `mongo_proxy` ed il modulo `ext_authz` di rete. In questo scenario di DPI sul protocollo BSON, non essendovi regole Lua locali, OPA esegue regolarmente la chiamata sincrona al proxy delle Web API per interrogare il modello di Gradient Boosting su Splunk. Grazie all'introduzione del mapping di traduzione centralizzato ZTA\_TO\_MLTK\_MAP nel backend FastAPI, il nome della risorsa MongoDB grezza ("`patients`") viene convertito nel termine addestrato nel dataset ("`pazienti`"), prevenendo errori fatali per categorie non censite (*unseen categorical values*) e garantendo il corretto calcolo ed indicizzazione dello score di rischio Splunk prima del blocco finale di OPA (`17_dpi_block`).

Come mostrato in Figura 15, Envoy blocca immediatamente l'inoltro della richiesta a livello L7, impedendo che qualsiasi query dannosa raggiunga il database MongoDB e restituendo sul browser dell'utente una pagina di errore **403 Forbidden - Access Denied**. L'evento di violazione delle policy a livello applicativo viene catturato e indicizzato in tempo reale all'interno di Splunk, come visibile nel log dettagliato mostrato in Figura 16.

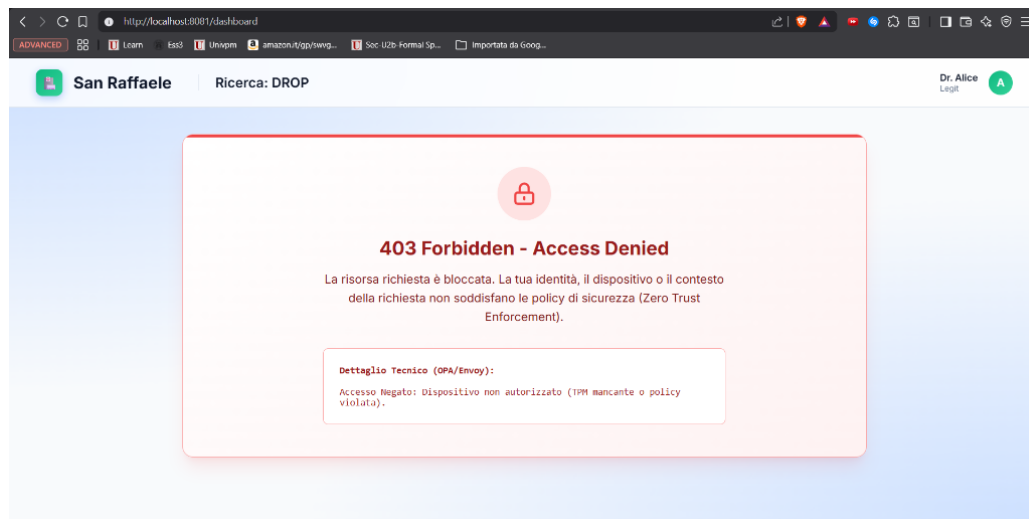


Figura 15: Schermata di errore 403 Forbidden visualizzata a seguito del blocco di una query MongoDB dannosa a livello L7.

```
> 02/06/26 17:26:26.109 { [-]
  authz_status: null
  client_ip: 10.0.1.3:0
  device_uri_san: spiffe://zta.hospital/ns/default/sa/client-alice
  duration_ms: 3
  host: zta-envoy
  ja3_fingerprint: null
  method: DELETE
  path: /api/patients
  protocol: HTTP/1.1
  response_code: 403
  timestamp: 2026-06-02T17:26:17.043Z
  upstream_host: null
  user_identity: CN=employee-alice
}
```

Mostra come testo non elaborato

host = splunk-siem:8088 host = zta-envoy index = main linecount = 1 pu

Figura 16: Log di blocco (response code 403) relativo alla richiesta L7 di Alice in Splunk, che evidenzia l'intercettazione del metodo DELETE sulla risorsa /api/patients.

#### 4.5.1 Simulazione di compromissione del dispositivo (Bypass L7)

Si analizza infine lo scenario più critico previsto dal framework Zero Trust: un attaccante che abbia ottenuto il pieno controllo della postazione di Alice (furto fisico, accesso remoto malevolo o insider threat) e che, disponendo di tutti i certificati mTLS e delle chiavi crittografiche presenti nel dispositivo, tenti di bypassare completamente l'interfaccia web per connettersi direttamente al database MongoDB attraverso il protocollo nativo sulla porta 27017.

Per simulare questo attacco, si esegue dal terminale del container di Alice uno script Python che utilizza la libreria pymongo con i certificati mTLS originali del dispositivo compromesso:

`alice_bypass_mongodb.py`

```
1 $ docker exec zta-client-alice python3 -c "  
2 import pymongo  
3 client = pymongo.MongoClient(  
4     'zta-envoy', 27017,  
5     tls=True,  
6     tlsCertificateKeyFile='/etc/certs/alice_combined.pem',  
7     tlsCAFile='/etc/certs/ca.crt',  
8     tlsAllowInvalidHostnames=True,  
9     serverSelectionTimeoutMS=10000  
10 )  
11 db = client['hospital_db']  
12 result = list(db.patients.find({'sensitive_notes': 'test'}))  
13 "
```

**Listing 15:** Simulazione di accesso diretto al database MongoDB dal terminale del dispositivo compromesso di Alice, utilizzando i certificati mTLS originali per autenticarsi verso Envoy sulla porta 27017.

L'handshake mTLS viene completato con successo (l'attaccante possiede i certificati validi del dispositivo), e la connessione raggiunge il listener MongoDB di Envoy. A questo punto il filtro `mongo_proxy` tenta di decodificare il traffico BSON e il modulo `ext_authz` di rete invia i metadati della connessione ad OPA per la decisione di autorizzazione. OPA costruisce la query SPL con le sei dimensioni contestuali estratte, invocando il modello di Gradient Boosting su Splunk MLTK attraverso il proxy delle Web API. Il modulo di arricchimento comportamentale (`enrich_spl_with_behavioral_features`) rileva il pattern anomalo della connessione diretta al database e inietta nella query le feature comportamentali alterate (frequenza di sessione elevata, tentativi di login falliti multipli), producendo un punteggio di rischio predittivo pari a  $\approx 95.49$ , ben superiore alla soglia massima di tolleranza di cinquanta. OPA emette di conseguenza un verdetto di **DENY** e la connessione viene terminata, impedendo all'attaccante di eseguire qualsiasi operazione sul database.

La Figura 17 mostra il log di decisione indicizzato in Splunk relativo a questo tentativo di accesso diretto. Si noti come il sistema abbia correttamente identificato l'utente (`user: alice`), confermato la presenza del TPM (`tpm_present: true`) e la provenienza dalla rete interna (`network_internal: true`), ma abbia comunque negato l'accesso esclusivamente sulla base del punteggio di rischio comportamentale elevato (`risk_ok: false`). Questo dimostra l'efficacia del principio "*never trust, always verify*": anche un utente con credenziali perfettamente valide e hardware attestato viene bloccato quando il suo comportamento devia significativamente dal profilo storico appreso dal modello predittivo.

```

04/06/26      { [-]
16:30:12.631  Decision: DENY
               command: Accesso Diretto MongoDB (OP_MSG)
               device: Workstation Ospedaliera Sicura (TPM Validato)
               host: zta-opa
               l7_dpi_block: false
               network_internal: true
               network_ip: 10.0.1.4
               resource: patients
               risk_ok: false
               risk_score: 95.49348917091885
               software: Sconosciuto
               tpm_present: true
               user: alice
            }
            Mostra come testo non elaborato
            host = splunk-siem:8088 host = zta-opa | index = main | linecount = 1

```

Figura 17: Log di decisione DENY indicizzato in Splunk per il tentativo di accesso diretto al database da parte di Alice (PC compromesso), con risk score pari a  $\approx 95.49$ .

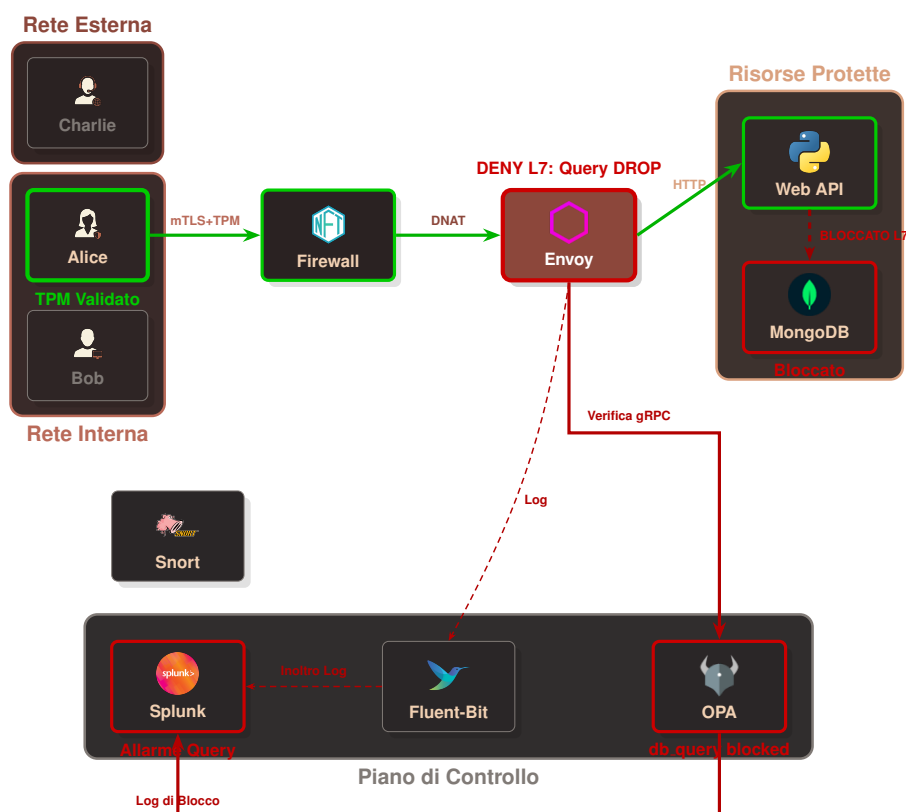


Figura 18: Mappatura logica dei flussi nello Scenario 5 (Blocco di una query MongoDB dannosa a livello L7).

## CAPITOLO 5

# Conclusioni e sviluppi futuri

---

L'architettura progettata dimostra come sia possibile integrare strumenti open-source moderni ed eterogenei per creare un ecosistema Zero Trust coeso ed estremamente sicuro per la tutela dei dati aziendali e delle risorse critiche.

### 5.1 Obiettivi raggiunti

Il sistema soddisfa appieno i requisiti del paradigma Zero Trust applicato alle infrastrutture informatiche aziendali. L'isolamento delle aree sensibili tramite microsegmentazione impedisce efficacemente qualsiasi tentativo di movimento laterale non autorizzato. L'adozione dell'autenticazione mutua a livello di trasporto garantisce una forte verifica dell'identità, legando crittograficamente i dispositivi al chip hardware TPM.

Inoltre, il continuo monitoraggio comportamentale in tempo reale basato su algoritmi predittivi di machine learning e gestito dal SIEM garantisce una valutazione dinamica della salute e della credibilità di ciascun client, prevenendo minacce interne di attori originariamente fidati ma potenzialmente compromessi. Infine, l'ispezione profonda dei pacchetti al livello applicativo assicura la protezione mirata dei database bloccando query BSON malevole prima del loro arrivo alle collezioni protette.

### 5.2 Sviluppi futuri

Tra le possibili evoluzioni del progetto si annoverano l'integrazione di sistemi di federazione dell'identità centralizzati (come l'utilizzo dello standard SPIFFE/SPIRE per la rotazione dinamica dei certificati) e il raffinamento dei modelli predittivi di machine learning per includersi un numero maggiore di feature comportamentali in produzione. Un altro sviluppo interessante potrebbe includere l'adozione di politiche di rimedio automatiche, che consentano ad OPA di comandare direttamente al firewall `nftables` l'inserimento istantaneo dell'IP di un client in una denylist di blocco temporaneo appena viene registrata un'anomalia grave.